HewaSense — System Naming & Architecture Reference

Purpose of this document

This refined reference captures the authoritative naming, architecture, and operational workflow for HewaSense. It incorporates the solution workflow diagram (Data Processing → Risk Modeling → Insight Delivery) and updates the system documentation to reflect operational details, alerting flow, risk triggers, and versioning conventions.

Public system name

HewaSense

HewaSense is the public-facing name of the climate intelligence platform. The name combines 'Hewa' (Swahili: air / atmosphere) and 'Sense' (detection, interpretation, intelligence). HewaSense focuses on monitoring and interpreting atmospheric signals for drought, flood, and crop-failure risk in Tanzania.

Workflow diagram (context)

The workflow diagram below illustrates the three primary system pillars and data flow:

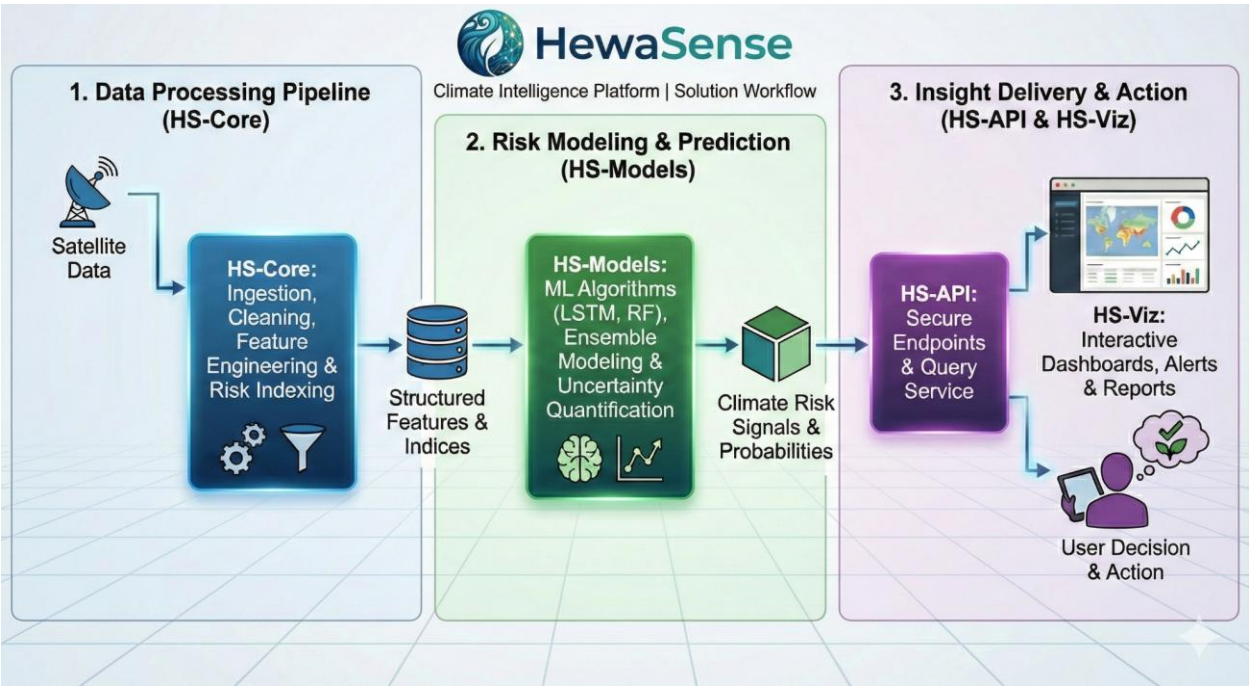

Figure 1 — HewaSense solution workflow: (1) Data Processing Pipeline (HS-Core) → (2) Risk Modeling & Prediction (HS-Models) → (3) Insight Delivery & Action (HS-API & HS-Viz).

Key pillars (detailed)

1. HS-Core — Data Processing Pipeline

HS-Core performs satellite-only ingestion, quality control, feature engineering, and risk indexing. It is explicitly the single source of truth for structured features and indices used by downstream models.

Primary responsibilities:

• Satellite data ingestion (multiple sources: precipitation, surface temperature, soil moisture proxies, vegetation indices).

• Reanalysis dataset integration for temporal continuity and bias correction.

• Quality assurance (range checks, cloud/qa flags handling, gap-aware interpolation rules).

• Feature engineering: rolling aggregates (7-, 30-, 90-day windows), seasonality encodings (sine/cosine), standardized anomalies, and spatial aggregations per administrative unit.

• Risk index computation and persistence: drought indices (e.g., standardized rainfall percentiles, SPEI-like proxies), flood indices (short-window extreme accumulation and intensity metrics), and crop-failure indicators (phenology-aware stress flags using vegetation indices and heat/ rainfall timing).

• Provenance & versioning: every feature and index must carry source, processing date, and HS-Core version tag.

2. HS-Models — Risk Modeling & Prediction

HS-Models ingests standardized features from HS-Core and produces probabilistic climate risk signals and predictions.

Primary responsibilities:

• Implement baseline models (Random Forest, Ridge) for interpretability and fast iteration; maintain training pipelines and hyperparameter records.

• Implement multivariate and sequence-aware models (LSTM / temporal CNN) to capture cross-variable and lagged dependencies.

• Ensemble and stacking strategies to improve robustness and calibration.

• Produce uncertainty-aware outputs: prediction intervals, calibrated probabilities, and per-region confidence scores.

• Model validation: time-aware cross-validation, region-based holdouts, and backtesting against historical events.

3. HS-API & HS-Viz — Insight Delivery & Action

HS-API exposes prediction endpoints and alert triggers; HS-Viz provides dashboards, downloadable reports, and user-facing alerts. Together they close the loop from signal to decision.

Primary responsibilities:

• HS-API: secure endpoints for queries, batch prediction jobs, and alert subscription management (webhooks/SMS/email).

• HS-Viz: interactive maps, time-series exploration, risk dashboards, contextual explanations, and exportable summaries (CSV/JSON/PDF).

• Alerting workflow: HS-Models writes risk signals → HS-API evaluates trigger logic → HS-API publishes alerts to HS-Viz and configured channels (webhook, SMS, email).

• Low-bandwidth modes: lightweight exports, SMS-friendly summaries, and snapshot reports for field use.

Risk triggers & example logic

Triggers should be framed as probabilistic, auditable rules combining multiple inputs. Example templates (customize thresholds to local context):

• Drought Trigger (example): 30-day rainfall percentile < 20% AND 30-day average temperature anomaly > +1.5σ ⇒ Issue 'Drought Watch' with probability score from HS-Models.

• Flood Trigger (example): 3-day accumulated rainfall > 95th percentile OR short-window intensity (e.g., hourly or daily peak) above local intensity threshold ⇒ Issue 'Flood Alert'.

• Crop-Failure Trigger (example): During defined crop critical window, vegetation index (e.g., NDVI/EVI) decline > 20% from long-term mean AND rainfall deficit during same window ⇒ Flag 'Crop Failure Risk'.

Operational considerations: triggers must include confidence thresholds, cooldown windows (to avoid alert fatigue), and human review for high-impact alerts.

Data & artifact versioning

Versioning conventions ensure reproducibility and traceability across HS-Core and HS-Models:

• HS-Core: hs-core-vYYYY.MM.patch (e.g., hs-core-v2025.07.1) — increments when ingestion, QA, or feature logic changes.

• HS-Models: hs-models-exp<tag>-vX (e.g., hs-models-expLSTM-v03) — experiment tags for tracking. Productionized ensembles receive semver-style versions.

• Dataset snapshots: data-yyyy-mm-dd.zip or data-YYYYMMDD.parquet stored in /data/snapshots with manifest files listing sources and checksums.

Code, repo & naming conventions

Recommended repository layout (consistent with prior doc):

```
hewasense/
├── hs-core/
├── hs-models/
├── hs-api/
├── hs-viz/
├── data/
├── docs/
└── README.md
```

Suggested internal naming patterns: HS-Core, HS-Models, HS-API, HS-Viz for modules; use hs- prefix for repository folders and package names. Use lowercase dash-separated file names for scripts (e.g., hs-core-ingest.py).

Appendix: Operational notes

• Provenance: store processing logs and manifest for every pipeline run.

• Monitoring: instrument HS-Core and HS-API with basic health metrics and error logs; track data latency and model drift metrics in HS-Models.

• Security & access: protect HS-API endpoints with API keys and role-based access for exports containing sensitive aggregated data.

• Pilot suggestions: run a 3-month pilot in selected districts with SMS alerts and feedback collection to calibrate triggers and UI wording.